WEBSITE DESIGN & DEVELOPMENT

MASTER PRODUCTION BRIEF — 2026

1. PROJECT OBJECTIVE

Build a premium, production-ready, scalable, responsive website with a powerful CMS that allows the client to manage all normal website and product content without developer intervention.

The website must feel like a professionally designed premium brand website — NOT like a generic AI-generated website.

The final result must prioritize:

- Premium visual design
- Strong typography
- High-quality product presentation
- Clean spacing and layout
- Fast performance
- Excellent mobile experience
- SEO
- Accessibility
- Security
- Easy content management
- Scalable product management
- Maintainable production-grade code
- Clear separation between content, design system and application logic

The supplied "Website brief 2026.pdf" is the primary client requirement document. Preserve all requirements from that document while implementing the more detailed technical requirements below.

---

2. IMPORTANT IMPLEMENTATION RULE

Do not blindly generate the website from this brief and stop.

Before implementation:

1. Analyze this entire specification.
2. Inspect the supplied reference websites.
3. Create the information architecture.
4. Create the component/design system.
5. Create the CMS content model.
6. Create the database/content relationships.
7. Create the route structure.
8. Create the responsive behaviour.
9. Create the SEO architecture.
10. Create the security architecture.
11. Then implement the complete website.

Do not ask unnecessary questions for values that can safely remain configurable through the CMS.

Where the brief says TBD, create a sensible CMS/configuration field and use clearly identifiable placeholder content until the real value is supplied.

Do not invent real business information, addresses, phone numbers, social links, product specifications or brand claims.

---

3. RECOMMENDED PRODUCTION TECH STACK

Frontend

Use:

- Next.js
- React
- TypeScript
- Next.js App Router
- Server Components by default
- Client Components only where interactivity requires them
- Modern CSS architecture
- CSS variables/design tokens
- Accessible semantic HTML

Use the latest stable versions that are mutually compatible at implementation time rather than unnecessarily pinning outdated versions.

Next.js is the preferred frontend framework because it supports React-based full-stack applications and the App Router provides the modern routing/server-component architecture.

---

4. CMS / BACKEND ARCHITECTURE

Use:

Payload CMS

Payload should be the CMS/backend layer for this project.

Reason:

- Native Next.js integration
- Production-ready admin panel
- TypeScript
- Authentication
- Access control
- REST API
- GraphQL support if needed
- Media management
- Image resizing/cropping
- Draft/publish capabilities
- Custom admin components
- PostgreSQL support
- Database migrations
- Extensible content models

Do NOT create a second unnecessary CMS.

Do NOT build a custom admin panel from scratch unless a specific requirement cannot be handled cleanly through Payload.

The architecture should be:

Browser
↓
Next.js
↓

Payload CMS / application backend
↓
PostgreSQL
↓
Object storage/CDN

Use PostgreSQL as the primary production database.

---

5. DATABASE

Use:

- PostgreSQL
- UUID primary identifiers where appropriate
- Proper foreign keys
- Database constraints
- Indexes for frequently queried fields
- Transactions for critical operations
- Migrations
- No unstructured database design

The database must support future catalogue growth without requiring architectural changes.

Recommended entities:

- Users
- Products
- Categories
- Product Images
- Product Downloads
- Product Relationships
- Pages
- Navigation
- Media
- SEO metadata
- Site Settings
- Design Settings
- Contact Settings

- Social Links
- Redirects
- Form Submissions
- Audit information where appropriate

PostgreSQL is the required relational database because the project contains structured products, categories, relationships, metadata and CMS content. PostgreSQL currently provides strong indexing, constraints, JSON/JSONB, transactions and other capabilities appropriate for this architecture.

---

6. MEDIA STORAGE

Do not depend on local server storage for production media.

Use production object storage such as:

- AWS S3
  OR
- Cloudflare R2
  OR
- another S3-compatible production object-storage provider.

Use a CDN in front of production assets where appropriate.

Images must support:

- WebP
- AVIF where practical
- Responsive sizes
- Automatic optimization
- Lazy loading
- Proper caching
- Focal point/cropping
- Alt text
- Metadata

---

7. EMAIL

Use a production transactional email provider or SMTP service.

Possible implementation:

- Resend
- Amazon SES
- Postmark
- SMTP provider supplied by client

Do not hard-code email credentials.

Use environment variables.

Contact-form emails must be tested in production/staging.

---

8. MONITORING

Production monitoring should include:

- Error monitoring
- Application logs
- Server/runtime logs
- Uptime monitoring
- Performance monitoring

Use a production monitoring service such as Sentry where appropriate.

Do not expose secrets, personal data or sensitive form information in logs.

---

9. PROJECT STRUCTURE

Use a clean, scalable architecture.

Suggested structure:

/app
/components
/features
/lib
/hooks
/types
/styles
/config
/payload
/public

/tests
/e2e

Organize components by responsibility rather than putting the entire application into one huge components folder.

Use reusable components.

Avoid duplicated code.

Avoid huge monolithic files.

---

10. WEBSITE PAGE STRUCTURE

The website must contain these primary page types:

PAGE 01 — HOME

Single-page scrolling homepage containing:

- Header/navigation
- Hero
- Brand/content sections according to final design
- Product/category sections where required
- Media sections where required
- CTA sections where required
- Footer

All appropriate content must be editable through CMS.

The homepage reference supplied in the original brief is:

Flos Professional — Bespoke

Use it as design/inspiration reference only.

Do not copy its content, branding or design literally.

---

11. HOME PAGE CMS STRUCTURE

Do not hard-code the homepage content.

Create reusable CMS-controlled sections.

The administrator should be able to:

- Add/edit section content

- Change headings
- Change text
- Change images
- Change videos
- Change buttons
- Change links
- Select products
- Reorder selected products
- Show/hide sections where appropriate

Recommended section architecture:

- Hero
- Intro/brand section
- Featured products
- Product categories
- Image/video storytelling section
- CTA
- Additional flexible sections
- Footer

The final visual layout must remain controlled by the design system rather than allowing the CMS to destroy the design.

---

12. PAGE 02 — PRODUCT SYSTEM

Create ONE reusable product template.

Never create separate hard-coded pages for individual products.

When an administrator creates:

Product
→ Information
→ Images
→ Downloads
→ SEO
→ Publish

the system must automatically create the product URL/page.

Example:

/products/product-name

No developer intervention should be required.

There must be no fixed product limit.

---

13. PRODUCT DATA MODEL

Each product should support at minimum:

Basic Information

- Product name
- Product slug
- Product code/SKU
- Short description
- Full description
- Status
- Featured flag

Classification

- Category
- Subcategory
- Tags

Product Media

- Main image
- Gallery images
- Image ordering
- Image alt text
- Image captions where required

Technical Information

Create a flexible technical-specification system rather than hard-coding a fixed number of fields.

Example:

Specification:

- Label
- Value

- Unit
- Group/order

This allows future product types to have different technical information.

Downloads

Support:

- PDF
- Technical documents
- Catalogues
- Datasheets
- Other approved file formats

Each download should have:

- Title
- File
- File type
- Optional description
- Optional version/date

Related Products

Allow administrators to manually select related products.

Optionally support automatic related-product suggestions based on category/tags, but manual control must remain available.

---

14. PRODUCT ADMIN FEATURES

The administrator must be able to:

- Add
- Edit
- Duplicate
- Delete
- Publish
- Unpublish
- Hide
- Show

- Reorder
- Feature/unfeature
- Categorize
- Add/remove images
- Reorder images
- Add/remove downloads
- Add related products
- Edit SEO
- Preview before publishing

Use confirmation dialogs for destructive actions.

Prevent accidental deletion.

---

15. PRODUCT CATALOGUE

Create a scalable product catalogue.

Recommended functionality:

- Product grid
- Product categories
- Category filtering
- Search
- Sorting
- Pagination or efficient infinite loading
- Empty states
- No-result states
- Product cards
- Product detail pages
- Breadcrumbs

Search should support at minimum:

- Product name
- Product code/SKU
- Relevant searchable metadata

Filtering should be CMS-driven where practical.

Do not create filters that are permanently hard-coded to one product type.

---

16. PRODUCT URL MANAGEMENT

Product URLs must be SEO-friendly.

Default:

/products/[slug]

Rules:

- Slugs must be unique
- Admin should be able to edit slug
- Duplicate slugs must be prevented
- Changing a published slug should support a redirect from the previous URL
- Deleted products should not create broken internal links
- Unpublished products should not appear publicly
- Canonical URL must be configurable
- No accidental duplicate URLs

---

17. PAGE 03 — CONTACT

Create a dedicated contact page.

Content:

- Address
- Email
- Telephone
- Social links
- Contact form

All appropriate information must be CMS editable.

Original brief reference:

Onok Lighting contact page.

Use only as inspiration.

---

18. CONTACT FORM

Create a production-grade contact form.

Recommended fields:

- Name
- Email
- Phone
- Company
- Subject
- Message
- Optional product selection
- Consent checkbox where legally required

Clearly identify:

- Required fields
- Optional fields

Implement:

- Client-side validation
- Server-side validation
- Spam protection
- Rate limiting
- Email notification
- Success state
- Error state
- Loading state
- Accessible validation messages

Never trust client-side validation alone.

Store submissions only if required by the business.

If submissions are stored, protect them through CMS access control.

---

19. NAVIGATION

Create responsive navigation based on the supplied final design.

Support:

- Logo
- Main navigation
- Product navigation
- Dropdown/mega menu if required
- CTA
- Social links where appropriate
- Mobile menu

Navigation content must be CMS editable where applicable.

Header behaviour must be intentionally designed.

Possible behaviour:

- Transparent over hero
- Solid after scroll
- Sticky header
- Mobile drawer

Do not assume these behaviours blindly; implement according to the final approved design.

---

20. FOOTER

Footer should support:

- Logo
- Navigation
- Contact information
- Social links
- Important links
- Legal links
- Copyright
- Optional newsletter/CTA if required

All business information must be CMS editable.

---

21. DESIGN SYSTEM

Create a centralized design system.

Do not scatter hard-coded design values throughout the application.

Use design tokens for:

- Colors
- Typography
- Font weights
- Spacing
- Border radius
- Shadows
- Container widths
- Breakpoints
- Motion
- Buttons
- Forms
- Cards

---

22. TYPOGRAPHY

CMS/design settings must support:

Fonts

- Primary font
- Secondary font
- Accent font

Values are currently TBD.

Fonts must be changeable without modifying application code where technically practical.

Font weights

Support:

- Thin
- Extra Light
- Light

- Regular
- Medium
- Semi Bold
- Bold

Typography levels

Support configurable:

- H1
- H2
- H3
- H4
- H5
- H6
- Body
- Small text
- Caption
- Navigation
- Buttons
- Product titles
- Product descriptions
- Technical information

Also support:

- Line height
- Letter spacing
- Text colour
- Alignment

---

23. GLOBAL COLOR SYSTEM

Create centralized color tokens.

Required configurable values:

- Primary background
- Secondary background

- Primary text
- Secondary text
- Accent
- Border
- Button background
- Button text
- Hover
- Active

Changing a global token should update every component using that token.

Do not hard-code the same HEX values repeatedly throughout the application.

---

24. BUTTON SYSTEM

Buttons must use reusable components.

Configurable properties:

- Text
- Font
- Font size
- Weight
- Text color
- Background
- Border
- Hover
- Active
- Disabled state

Support reusable variants such as:

- Primary
- Secondary
- Ghost
- Text/link
- Dark/light variants if required by design

---

25. RESPONSIVE DESIGN

Design intentionally for:

- Desktop
- Laptop
- Tablet
- Mobile

Do NOT simply shrink desktop.

Specifically adapt:

- Typography
- Navigation
- Images
- Product grids
- Spacing
- Buttons
- Animations
- Layout
- Horizontal scrolling
- Forms

Test real mobile viewport sizes.

---

26. ACCESSIBILITY

Build accessibility into the application from the beginning.

Requirements:

- Semantic HTML
- Keyboard navigation
- Visible focus states
- Accessible buttons
- Accessible links
- Proper form labels
- Error announcements where required

- Meaningful alt text
- Sufficient colour contrast
- Accessible navigation
- Accessible mobile menu
- Reduced-motion consideration
- Logical heading hierarchy
- No keyboard traps

Target WCAG 2.2 AA where reasonably applicable.

---

27. IMAGES

CMS must support:

- Upload
- Replace
- Delete
- Reorder
- Product assignment
- Homepage section assignment
- Alt text
- Captions where required

Implement:

- Responsive image sizes
- Automatic optimization
- Lazy loading
- Priority loading for critical hero imagery
- Modern formats
- CDN delivery
- Proper cache headers

Do not load massive original images when a smaller responsive version is sufficient.

---

28. VIDEO

If video is included in the final design, support:

- Upload/embed
- Replace
- Poster image
- Autoplay where specified
- Muted autoplay where specified
- Mobile fallback
- Responsive sizing
- Lazy loading where appropriate

Video must not unnecessarily compromise Core Web Vitals.

Video locations are currently TBD.

---

29. SEO

Every page must have configurable SEO metadata.

Support:

- Page title
- Meta description
- URL slug
- Canonical URL
- Image alt text
- Open Graph title
- Open Graph description
- Open Graph image

Products must automatically receive SEO-friendly URLs.

Implement:

- XML sitemap
- Robots.txt
- Proper heading hierarchy
- Canonical tags

- SEO-friendly URLs
- Metadata generation
- Open Graph
- Twitter/X metadata where useful
- 404 handling
- Redirect handling

---

30. STRUCTURED DATA

Implement appropriate JSON-LD/schema markup.

Potential schemas:

- Organization
- WebSite
- Product
- BreadcrumbList

Only output schemas that accurately describe the page.

Never generate fake ratings, prices, reviews or business information.

---

31. PERFORMANCE

Optimize for real-world performance and Core Web Vitals.

Implement:

- Server rendering where beneficial
- Static generation/revalidation where appropriate
- Optimized images
- Responsive image loading
- Lazy loading
- Optimized fonts
- Minimal JavaScript
- Code splitting
- Efficient components
- Caching

- Compression
- CDN
- Minimal third-party scripts
- Proper preloading only where justified

Do not add unnecessary libraries.

Do not add unnecessary animation libraries.

Every dependency must have a reason.

---

32. PERFORMANCE TARGET

Before final delivery, test using Lighthouse and appropriate real-device/browser testing.

Target:

- Excellent Core Web Vitals
- No obvious layout shift
- Fast first meaningful render
- Fast interaction
- Optimized mobile experience

Provide a final performance report.

---

33. SEARCH ENGINE CRAWLABILITY

Ensure:

- Public pages are crawlable
- Draft/unpublished content is not publicly indexable
- No accidental noindex on production pages
- Canonical URLs are correct
- Sitemap contains appropriate public URLs
- Robots.txt is correct
- Redirects are correct

---

34. CMS ADMIN EXPERIENCE

The CMS must be simple enough for a non-developer.

Dashboard should provide clear areas for:

- Products
- Categories
- Media
- Homepage
- Pages
- Navigation
- Site settings
- SEO
- Contact settings
- Form submissions, if stored
- Users/admins

Avoid unnecessary technical fields for ordinary content editors.

Use helpful field labels and descriptions.

---

35. CMS ROLES & PERMISSIONS

Support role-based access.

Recommended:

Super Admin

Full access.

Content Manager

Can manage:

- Pages
- Homepage
- Products
- Media
- SEO
- Navigation

But cannot modify critical system configuration unless explicitly allowed.

Editor

Can manage approved content areas.

All access should follow least-privilege principles.

---

36. CONTENT WORKFLOW

Support:

- Draft
- Preview
- Publish
- Unpublish
- Archive where appropriate

Where supported, provide:

- Revision history
- Restore previous version
- Publish scheduling if useful

Never expose draft content publicly.

---

37. SECURITY

Production implementation must include:

- HTTPS/SSL
- Secure cookies
- Secure authentication
- Strong password policy
- Rate limiting
- Spam protection
- Input validation
- Output sanitization where required
- CSRF protection where applicable
- Security headers

- Secure CORS configuration
- Dependency auditing
- Protected admin routes
- Database access controls
- Secret management through environment variables
- No secrets committed to Git
- Secure file upload validation

Use current secure defaults of the selected frameworks.

---

38. AUTHENTICATION

CMS administrator authentication must include:

- Secure login
- Logout
- Password reset
- Session management
- Account protection
- Rate limiting
- Optional/recommended 2FA for administrators

Do not build custom authentication unnecessarily when the CMS provides secure authentication.

---

39. BACKUPS

Production backups must include:

- Database
- Media
- Critical application configuration

Recommended:

- Automated backups
- Off-site backup storage
- Backup retention policy
- Restore testing

- Documented recovery process

Backup frequency is currently TBD by client, but the system must be designed to support automated scheduled backups.

---

40. HOSTING & DEPLOYMENT

Use a production-capable cloud architecture.

Preferred options:

- Vercel for Next.js where compatible
- AWS for infrastructure requiring greater control
- Cloudflare for CDN/DNS/security where appropriate

Do not select infrastructure simply because it is popular.

Choose based on:

- Performance
- Reliability
- Security
- Cost
- Scalability
- Maintainability
- Client ownership

Use separate environments:

- Local development
- Staging
- Production

Never develop directly against production.

---

41. ENVIRONMENT VARIABLES

Use environment variables for:

- Database URL
- CMS secret
- Authentication secrets

- Storage credentials
- Email credentials
- Analytics IDs
- Monitoring DSN
- Other third-party API keys

Provide:

.env.example

Never commit real secrets.

---

42. CI/CD

Set up automated CI/CD where practical.

Pipeline should include:

- Install
- Type checking
- Linting
- Unit tests
- Build
- E2E tests where configured
- Deployment

Production deployment should not occur if required checks fail.

---

43. TESTING

Implement automated testing appropriate to the application.

Unit tests

Test:

- Utility functions
- Validation
- Business logic
- Important data transformations

Integration tests

Test:

- CMS/database operations
- Product creation
- Product publishing
- Contact form
- SEO generation
- Media relationships

E2E tests

Test critical user journeys:

1. Homepage loads
2. Navigation works
3. Product catalogue works
4. Product detail page works
5. Product links work
6. Contact form works
7. Mobile navigation works
8. SEO metadata exists
9. CMS administrator can create a product
10. Product appears automatically after publishing

Use Playwright or an equivalent production-grade E2E framework.

---

44. BROWSER COMPATIBILITY

Test current supported versions of:

- Google Chrome
- Safari
- Microsoft Edge
- Firefox where practical

Operating systems:

- Windows
- macOS

- iOS
- Android

At minimum, Chrome and Safari are mandatory according to the original client brief.

---

45. ERROR HANDLING

Create proper:

- 404 page
- 500/error page
- Loading states
- Empty states
- Failed form state
- Failed image/media fallback
- CMS/API failure handling

Never expose stack traces or internal technical errors to public users.

---

46. ANALYTICS

Support:

- Google Analytics 4
- Google Search Console

Track meaningful events where appropriate:

- Product views
- Product downloads
- Contact form submission
- CTA clicks
- External link clicks
- Search usage

Do not add excessive tracking.

Analytics/account ownership must belong to the client.

---

47. PRIVACY & LEGAL

The implementation must provide locations for:

- Privacy Policy
- Cookie Policy where required
- Terms/Legal information
- Contact-form consent where required

Do not invent legal text.

Provide CMS-editable legal pages/placeholders.

If analytics/cookies require consent under applicable law, implement an appropriate consent mechanism.

---

48. SOCIAL MEDIA

CMS-editable:

- Instagram
- LinkedIn
- YouTube

Additional social platforms should be easy to add later.

Do not hard-code social URLs throughout components.

---

49. INTERNATIONALIZATION

Initial implementation should be:

- English
- Single-language

However, architecture should not make future multilingual support unnecessarily difficult.

Do not implement a complete multilingual system unless requested.

---

50. CONTENT RESPONSIBILITY

Clearly separate client-provided content from developer-created implementation.

Client must provide/finalize:

- Logo
- Brand assets
- Final copy
- Product data
- Product images
- Product technical information
- Contact details
- Social links
- Fonts/licenses where applicable
- Legal text
- SEO content where applicable

Developer is responsible for:

- Implementation
- CMS configuration
- Responsive UI
- Technical SEO
- Performance
- Security
- Testing
- Deployment
- Documentation
- Handover

Do not invent business information.

---

51. REFERENCE WEBSITES

Use the references supplied in the original PDF as visual/UX references.

Homepage:
Flos Professional — Bespoke

Product:
Flos Professional — Product/Subfamily

Contact:
Onok Lighting — Contact

IMPORTANT:

These are references, not templates to copy.

Do not copy:

- Branding
- Logos
- Text
- Product photography
- Proprietary illustrations
- Exact layouts
- Proprietary animations

Create an original design inspired by the quality, hierarchy and premium feel of the references.

---

52. VISUAL DESIGN DIRECTION

The website must NOT look like a generic AI-generated website.

Avoid:

- Excessive glassmorphism
- Random gradients
- Excessive rounded cards
- Generic SaaS dashboard styling
- Excessive shadows
- Random floating blobs
- Overuse of icons
- Template-like sections
- Excessive animation
- Artificial-looking decorative elements
- Stock-template layouts
- Excessive use of huge text without purpose

Prefer:

- Strong editorial composition
- Premium typography
- Generous whitespace

- Intentional grid
- High-quality imagery
- Sophisticated product presentation
- Restrained animation
- Clear hierarchy
- Consistent spacing
- Strong visual rhythm
- Professional navigation
- Excellent micro-interactions

The design should feel like it was produced by a professional digital design studio.

---

53. ANIMATION & INTERACTION

Use motion purposefully.

Possible:

- Subtle page transitions
- Image reveal
- Hover transitions
- Product-card interactions
- Navigation transitions
- Scroll-based effects where appropriate

Rules:

- Keep animation subtle
- Respect prefers-reduced-motion
- Do not sacrifice performance
- Do not animate everything
- Do not use animation merely to make the website appear “AI-generated”

---

54. CONTENT EDITABILITY

The following must NOT require source-code changes for ordinary updates:

Homepage

- Text
- Headings
- Images
- Videos
- Product selections
- Product ordering
- Buttons
- Links
- Sections

Products

- Name
- Description
- Images
- Information
- Downloads
- Related products
- SEO

Contact

- Address
- Email
- Phone
- Social media
- Contact-form settings

Global settings

- Fonts
- Colors
- Social links
- Contact information
- SEO defaults

---

55. NO HARDCODED BUSINESS CONTENT

Do not hard-code:

- Product names
- Product descriptions
- Phone numbers
- Email addresses
- Addresses
- Social URLs
- Product images
- SEO metadata
- Brand-specific configuration

Store these through CMS/configuration.

---

56. DATA VALIDATION

Validate:

- Product data
- Slugs
- Email addresses
- URLs
- File types
- File sizes
- Required fields
- SEO fields
- Contact form data

Validation must happen server-side as well as client-side where applicable.

---

57. FILE UPLOAD SECURITY

For uploads:

- Validate MIME type
- Validate extension
- Restrict file size
- Restrict dangerous formats
- Generate safe file names
- Do not execute uploaded files
- Store files outside executable application paths where appropriate
- Use secure object storage
- Validate images before processing

---

58. PRODUCT DELETE SAFETY

Deleting a product must not silently break:

- Related products
- Homepage selections
- Category pages
- URLs
- Images
- Downloads

Use appropriate relationship handling.

For published products, consider archive/unpublish before permanent deletion.

---

59. SEO REDIRECT SYSTEM

Create support for redirects.

At minimum:

- Old URL
- New URL
- Redirect type
- Active/inactive

Automatically create redirects when appropriate after changing published product slugs.

---

60. DATABASE PERFORMANCE

Create appropriate indexes for:

- Product slug
- Product code/SKU
- Category
- Published status
- Featured status
- Searchable fields
- Relationships

Do not perform unnecessary database queries.

Avoid N+1 queries.

Use caching/revalidation where appropriate.

---

61. CACHING

Use Next.js caching/revalidation strategically.

Public product/page content should be cacheable.

When CMS content changes:

- Relevant cached content must invalidate/revalidate
- Updated product pages must become visible without manual server intervention

Do not make every page dynamically uncached unless necessary.

---

62. ADMIN PREVIEW

Provide a practical content-preview workflow where supported.

The administrator should be able to preview:

- Homepage
- Product page
- Content pages

before publishing.

---

63. ADMIN UX

CMS must be designed/configured for non-technical users.

Use:

- Clear labels
- Helpful descriptions
- Logical grouping
- Required-field indicators
- Image previews
- Drag-and-drop ordering where useful
- Confirmation dialogs
- Helpful validation messages

Avoid exposing unnecessary technical complexity.

---

64. DOCUMENTATION

Final delivery must include:

Technical documentation

- Architecture
- Project structure
- Setup instructions
- Environment variables
- Database setup
- CMS setup
- Deployment process
- Backup process
- Restore process
- Email configuration
- Analytics configuration

CMS documentation

Explain how to:

- Add product
- Edit product
- Publish/unpublish product
- Upload images
- Add downloads
- Add related products
- Edit homepage
- Edit navigation
- Edit contact information
- Edit SEO
- Manage administrators

---

65. OWNERSHIP & HANDOVER

The client must receive full administrator access.

Developer must not retain exclusive access to:

- CMS
- Hosting
- Domain
- Database
- Source code
- Analytics
- Email service
- Storage
- Third-party services

At final handover provide:

- Complete source code
- Project files
- CMS access
- Database access
- Hosting access
- Domain/DNS access

- Storage access
- Analytics access
- Search Console access
- Email service access
- Third-party service access
- Documentation

Third-party software remains subject to its applicable license.

---

66. ACCOUNT OWNERSHIP

Where possible, production accounts should be created under client-owned email/account credentials.

Do not create critical production services under a developer's personal account.

The client should own:

- Domain
- Hosting
- Database
- Storage
- Analytics
- Search Console
- Email provider
- Monitoring account
- Third-party subscriptions

Developer can be added as a collaborator/admin where necessary.

---

67. STAGING ENVIRONMENT

Create a staging environment before production.

Staging must allow:

- CMS testing
- Product testing
- Form testing

- Responsive testing
- SEO testing
- Performance testing

Prevent staging from being indexed by search engines.

Do not send real production emails from staging without explicit configuration.

---

68. PRODUCTION LAUNCH CHECKLIST

Before launch verify:

Design

- Desktop approved
- Laptop approved
- Tablet approved
- Mobile approved
- All states designed

CMS

- Product creation works
- Product editing works
- Product publishing works
- Product deletion/archive works
- Homepage editing works
- Media management works
- SEO management works
- Navigation management works

Functionality

- Navigation works
- Product catalogue works
- Search works
- Filters work
- Product pages work
- Contact form works

- Email delivery works
- Social links work
- Downloads work

SEO

- Metadata
- Sitemap
- Robots.txt
- Canonicals
- Structured data
- Open Graph
- Product URLs
- Redirects
- Heading hierarchy

Performance

- Image optimization
- Font optimization
- Lazy loading
- Caching
- CDN
- Core Web Vitals
- Lighthouse testing

Security

- SSL
- Secure authentication
- Admin protection
- Rate limiting
- Spam protection
- Security headers
- Secrets protected
- Dependency audit
- Backups

Compatibility

- Chrome
- Safari
- Firefox where applicable
- Edge where applicable
- Windows
- macOS
- iOS
- Android

---

69. ACCEPTANCE CRITERIA

The project is considered technically complete only when:

1. All approved page types are implemented.
2. CMS functionality works without developer intervention.
3. New products can be created from the CMS.
4. New product pages are generated automatically.
5. Product URLs are SEO-friendly.
6. Product media works correctly.
7. Contact forms work.
8. Email delivery works.
9. Responsive layouts work.
10. SEO requirements are implemented.
11. Performance testing is completed.
12. Security requirements are implemented.
13. Browser/device testing is completed.
14. Production backups are configured.
15. Analytics/Search Console are configured.
16. Documentation is delivered.
17. Client access is transferred.
18. Source code is delivered.
19. No critical or high-severity known bugs remain.

20.Client approves the final production build.

---

70. OUT-OF-SCOPE DEFAULTS

Unless explicitly requested later, do NOT add:

- E-commerce checkout
- Online payments
- User/customer accounts
- Shopping cart
- Wishlist
- Product purchasing
- Complex ERP integration
- Inventory management
- AI chatbot
- Unnecessary animations
- Unnecessary third-party services
- Multilingual system
- Complex search infrastructure

The architecture should remain extensible for future additions.

---

71. CURRENTLY TBD VALUES

Do not invent these.

Create configurable placeholders:

- Expected delivery date
- Primary font
- Secondary font
- Accent font
- Initial font weights
- Initial font sizes
- Primary background HEX
- Secondary background HEX

- Primary text HEX
- Secondary text HEX
- Accent HEX
- Border HEX
- Button background HEX
- Button text HEX
- Hover HEX
- Active HEX
- Video locations
- Desktop breakpoint/specification
- Laptop breakpoint/specification
- Tablet breakpoint/specification
- Mobile breakpoint/specification
- Primary email
- Secondary email
- Instagram URL
- LinkedIn URL
- YouTube URL
- Backup frequency
- Final product fields
- Final homepage section content

Use clearly marked placeholder values in development.

---

72. DEVELOPMENT PRINCIPLES

Follow these rules throughout the project:

Code quality

- TypeScript
- Strict typing
- Reusable components
- Clear naming
- No unnecessary duplication

- No dead code
- No unnecessary dependencies
- No hard-coded business data
- No secrets in source code

Architecture

- Scalable
- Maintainable
- Modular
- Testable
- Secure
- SEO-friendly
- Performance-oriented

UI

- Premium
- Original
- Responsive
- Accessible
- Consistent
- Editorial
- Professional

CMS

- Easy for non-developers
- Flexible
- Safe
- Structured
- Scalable

---

73. FINAL ANTIGRAVITY INSTRUCTION

Treat this document as the master implementation specification.

Do not produce a superficial demo.

Build the actual production architecture.

Do not stop after creating the homepage.

The implementation must include:

1. Complete frontend
2. Complete CMS
3. Product system
4. Database
5. Media management
6. Contact system
7. SEO system
8. Responsive design
9. Accessibility
10. Security
11. Performance optimization
12. Analytics integration points
13. Testing
14. Deployment configuration
15. Documentation

Before considering the project complete, verify every requirement in this document against the actual implementation.

Create a final REQUIREMENTS-CHECKLIST.md containing every requirement and mark each one as:

- Implemented
- Pending client information
- Pending external credential/configuration
- Not applicable

Do not mark a requirement as complete unless it has actually been implemented and tested.

The final result must be a professional, production-ready website — not a prototype, mockup, static demo or AI-generated template.